

Torrent Analyze

⚠ **Security Warning:** Work in an isolated environment (virtual machine or container). Never open unknown files on your primary machine.

Challenge Description

SOS, someone is torrenting on our network.

One of your colleagues has been using torrent to download some files on the company's network. Can you identify the file(s) that were downloaded? The file name will be the flag, like picoCTF{filename}.

Hints

- Download and open the file with a packet analyzer like [Wireshark](#).
 - You may want to enable BitTorrent protocol (BT-DHT, etc.) on Wireshark. Analyze → Enabled Protocols
 - Try to understand peers, leechers and seeds. [Article](#)
-

Tools Used

- `file` – Determine file type
- `strings` – Extract printable character sequences
- `exiftool` – Read PCAP metadata
- **Wireshark** – GUI tool for analyzing network traffic
- `tshark` – Command-line Wireshark tool for packet analysis
- `grep` – Filter output for specific patterns
- `awk` – Text processing and field extraction
- `cut` – Extract specific fields from output
- `uniq` – Count and remove duplicate lines
- **Web browser** – Search torrent hash on torrent indexing sites

Complete Solution

Step 1: Download and Prepare

Download the PCAP file named `torrent.pcap` from the challenge page.

Step 2: Basic File Inspection

First, verify the file type:

```
file torrent.pcap
```

Expected output:

```
torrent.pcap: pcapng capture file - version 1.0
```

The file is a **PCAPNG** capture file containing network traffic.

Step 3: Search for Visible Text

A quick `strings` scan to see if anything jumps out:

```
strings torrent.pcap | grep -i "picoCTF"
```

Output: (no results)

The flag isn't directly visible. Time to analyze the torrent traffic.

Step 4: Examine Metadata

```
exiftool torrent.pcap
```

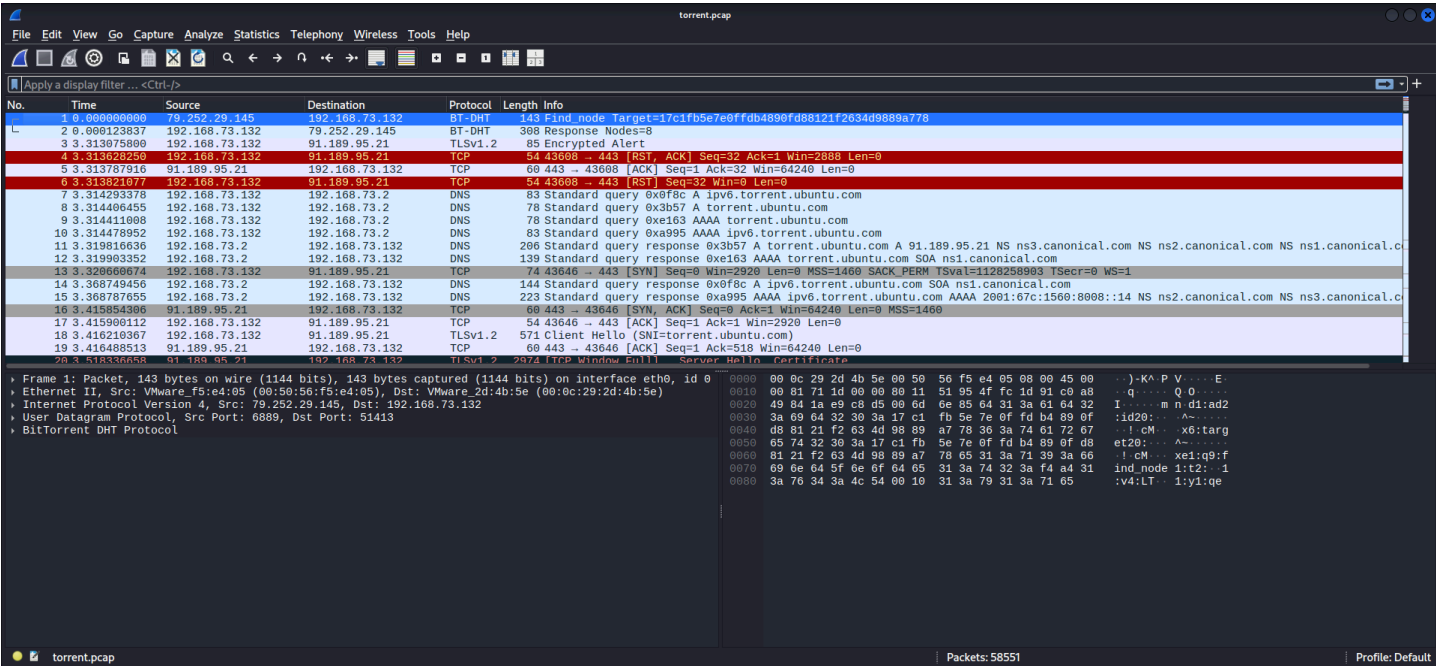
Output:

```
ExifTool Version Number      : 13.50
File Name                    : torrent.pcap
Directory                    : .
File Size                    : 64 MB
File Modification Date/Time  : 2026:05:19 21:35:20+02:00
...
User Application              : Dumpcap (Wireshark) 3.2.1
Operating System              : Linux 5.4.0-kali3-amd64
Time Stamp                    : 2020:02:17 22:00:42.052649021+02:00
```

Nothing suspicious in the metadata.

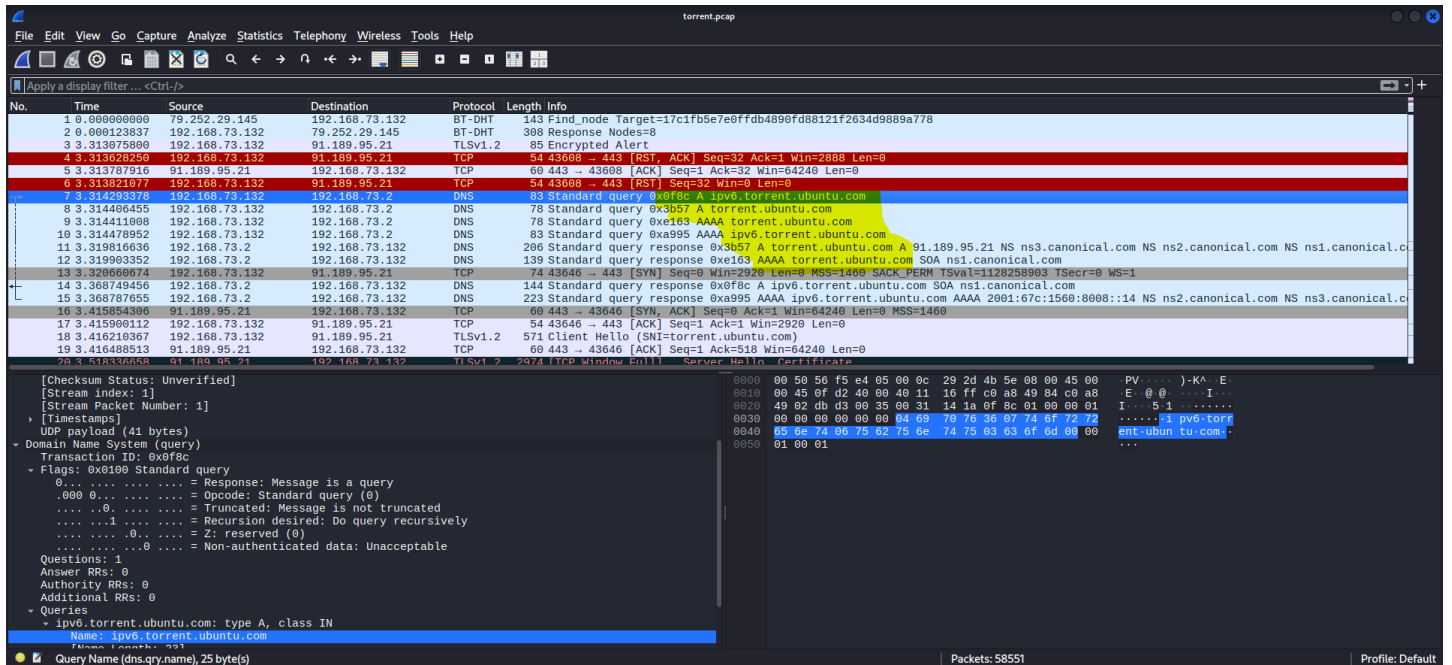
Step 5: Analyze with Wireshark (Visual Inspection)

Open the file in Wireshark:



The traffic contains several interesting subdomains related to torrenting:

- ipv6.torrent.ubuntu.com
- torrent.ubuntu.com



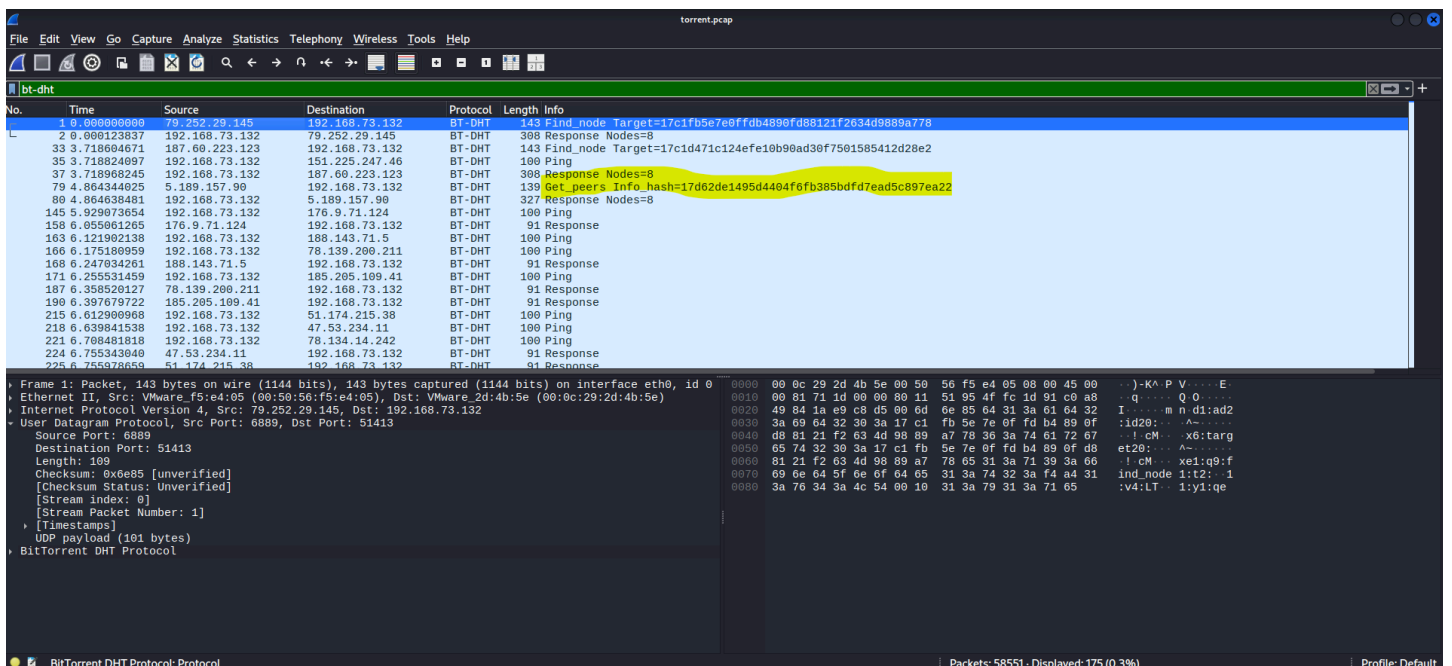
Visiting these websites shows many torrent hashes, but we need to find which one was actually downloaded.

Step 6: Understand BitTorrent DHT (Distributed Hash Table)

The hint mentions enabling **BT-DHT** in Wireshark (Analyze → Enabled Protocols). DHT is used by BitTorrent to find peers without a central tracker.

Key DHT messages:

- `get_peers` – A node asking for peers that have a specific torrent (identified by `info_hash`)
- `Info_hash` – A 40-character SHA-1 hash uniquely identifying a torrent file



The most frequently requested `info_hash` is likely the one being downloaded!

Step 7: Extract DHT Traffic with tshark

First, filter all BT-DHT packets:

```
tshark -nr torrent.pcap -Y "bt-dht"
```

Output (excerpt):

```
1 0.000000000 79.252.29.145 → 192.168.73.132 BT-DHT 143 Find_node
Target=17c1fb5e7e0ffdb4890fd88121f2634d9889a778
2 0.000123837 192.168.73.132 → 79.252.29.145 BT-DHT 308 Response Nodes=8
33 3.718604671 187.60.223.123 → 192.168.73.132 BT-DHT 143 Find_node
Target=17c1d471c124efe10b90ad30f7501585412d28e2
35 3.718824097 192.168.73.132 → 151.225.247.46 BT-DHT 100 Ping
37 3.718968245 192.168.73.132 → 187.60.223.123 BT-DHT 308 Response Nodes=8
79 4.864344025 5.189.157.90 → 192.168.73.132 BT-DHT 139 Get_peers
Info_hash=17d62de1495d4404f6fb385bdfd7ead5c897ea22
...
```

Now, filter only `Get_peers` messages (these requests ask for peers with a specific torrent):

```
tshark -nr torrent.pcap -Y "bt-dht" | grep -i "Get_peers"
```

Output (excerpt):

```
79 4.864344025 5.189.157.90 → 192.168.73.132 BT-DHT 139 Get_peers
Info_hash=17d62de1495d4404f6fb385bdfd7ead5c897ea22
429 8.605862029 186.105.1.103 → 192.168.73.132 BT-DHT 146 Get_peers
Info_hash=d59b1ce3bf41f1d282c1923544629062948afadd
436 8.771325275 191.37.246.111 → 192.168.73.132 BT-DHT 146 Get_peers
Info_hash=078e18df4efe53eb39d3425e91d1e9f4777d85ac
1587 13.891232052 212.96.69.84 → 192.168.73.132 BT-DHT 146 Get_peers
Info_hash=7af6be54c2ed4dcb8d17bf599516b97bb66c0bfd
3999 15.367739447 114.35.63.17 → 192.168.73.132 BT-DHT 157 Get_peers
Info_hash=17c0c2c3b7825ba4fbe2f8c8055e000421def12c
...
```

Step 8: Count Unique Info_hashes

Extract just the `Info_hash` values, count their occurrences, and find the most requested one:

```
tshark -nr torrent.pcap -Y "bt-dht" | grep -i "Get_peers" | awk '{print $9}' | cut -d '=' -f2 | uniq -c
```

Output:

```
1 17d62de1495d4404f6fb385bdfd7ead5c897ea22
1 d59b1ce3bf41f1d282c1923544629062948afadd
1 078e18df4efe53eb39d3425e91d1e9f4777d85ac
1 7af6be54c2ed4dcb8d17bf599516b97bb66c0bfd
1 17c0c2c3b7825ba4fbe2f8c8055e000421def12c
1 17c02f9957ea8604bc5a04ad3b56766a092b5556
29 e2467cbf021192c241367b892230dc1e05c0580e
```

Key finding: The hash `e2467cbf021192c241367b892230dc1e05c0580e` appears **29 times** – far more than any other. This is likely the torrent being downloaded!

Step 9: Search the Torrent Hash Online

Now we need to find what file this hash corresponds to.

Search the hash on torrent indexing websites:

- <https://cn.torrentkitty.net/information/E2467CBF021192C241367B892230DC1E05C0580E>

Result: The page reveals the ISO filename:

```
<REDACTED>-amd64.iso
```

Step 10: Format the Flag

The flag is the filename of the downloaded torrent, wrapped in `picoCTF{}` .

One-Liner Solution

Extract and count `Get_peers` requests in one command:

```
tshark -nr torrent.pcap -Y "bt-dht" | grep -i "Get_peers" | awk '{print $9}' | cut -d '=' -f2 | uniq -c | sort -rn
```

Final Result

picoCTF{<REDACTED>}

Note: The actual flag is redacted here. In the real challenge, searching the hash on a torrent index will reveal the filename.

Key Concepts

Concept	Description
BitTorrent DHT	A distributed hash table used by BitTorrent to find peers without a central tracker.
Info_hash	A 40-character SHA-1 hash that uniquely identifies a torrent file.
Get_peers	A DHT query asking for peers that have a specific torrent (identified by info_hash).
Leechers vs Seeds	Leechers download, seeds upload. High <code>get_peers</code> activity often indicates a popular or actively downloaded torrent.
BT-DHT Protocol	Enabled in Wireshark under "Enabled Protocols" to decode DHT traffic.
Torrent Indexing Sites	Websites that store torrent metadata, allowing you to search by info_hash.

Pro Tips

1. **Enable BT-DHT in Wireshark** – Go to Analyze → Enabled Protocols and enable “bt-dht” to see decoded DHT messages.
 2. **Info_hash length** – A valid SHA-1 hash is exactly 40 hexadecimal characters. Look for this pattern.
 3. **Count frequencies** – The most requested `info_hash` is likely the one being downloaded.
 4. **Search multiple torrent sites** – If one site doesn’t have the hash, try others like `linuxtracker.org` , `torrentkitty.net` , or `btdig.com` .
 5. **Check Ubuntu torrents** – The subdomains `torrent.ubuntu.com` suggest the download might be an Ubuntu ISO.
 6. **`uniq -c` shows counts** – This command prefixes each unique line with its occurrence count.
-

Alternative Torrent Search Tools

Tool	Command / Link
TorrentKitty	https://cn.torrentkitty.net/information/
Linux Tracker	https://linuxtracker.org/